Javascript : Fonctions (II)

Fonctions

mind-mapping: https://www.mindmeister.com/1901813655?t=IR3KxybQDr#

Définir des fonctions

Les déclarations de fonctions

Une définition de fonction (aussi appelée déclaration de fonction ou instruction de fonction) est construite avec le mot-clé function, suivi par :

- Le nom de la fonction.
- Une liste d'arguments à passer à la **fonction**, entre parenthèses et séparés par des virgules.
- Les instructions JavaScript définissant la fonction, entre accolades, { }.

```
 // function declaration
 function createCell(value, text) {
      var cell = document.createElement("td");
      cell.setAttribute("class", "detailsColumn");
      cell.dataset.value = value;
      cell.innerText = text;
      return cell;
  }

// function expression
const connectToDatabase = function( dbUri ){
    try{
       return dbUri.connect();
    }catch( error ){
       throw new Error('Erreur de connection à la base de donnée');
    }
}

// flat-arrow function
const connectToDatabase = ( dbUri )=>{
    try{
       return dbUri.connect();
    }catch( error ){
       throw new Error('Erreur de connection à la base de donnée');
    }
}

// OR
const connectToDatabase = dbUri => dbUri.connect();
```

- createCell : nom de la fonction
- value, text : arguments à passer à la fonction
- cell : la valeur de retour de la fonction

Execution d'une fonction (invoke, run, execute, call)

```
let cellule = createCell(1,"Liste de Clients");
```

les parenthèses réprensentent un operateur d'appel de fonction.

NB: une fonction peut ne pas avoir des arguments

```
function renderView(){
   // do drawing staff here
}
```

```
// function invocation
renderView();
```

Passage Par Valeur, Passage Par Référence

Les paramètres primitifs (comme les nombres) sont passés à la fonction par valeur. La valeur est passée à la fonction mais si cette dernière change la valeur du paramètre, cela n'aura pas d'impact au niveau global ou au niveau de ce qui a appelé la fonction.

Si l'argument passé à la fonction est un objet (une valeur non-primitive, comme un objet Array ou un objet défini par l'utilisateur), et que la fonction change les propriétés de cet objet, ces changements seront visibles en dehors de la fonction. Par exemple :

```
// if the type of value of myName is primitif
let myName="Ferid HELALI";

function transform(vlaue){
     console.log('Running transform funtion ...');
     value = "Salem BEN AMOR";
}

console.log(` myName before transform invocation : ${myName}`);
transform(myName);
console.log(` myName after transform invocation : ${myName}`);

/*
  myName still the same and not affected by the assaignment
  in the transform function this behavior is called the parameter
  is passed by value and this is true for all primitives
  Types ( number , string, boolean, undefined, null
*/



// if the type of value of myName is object
let student={
       id:1,
       username:"Ali OMRANE",
       gender: 'Male'}
    ;

function transform(value){
     console.log('Running transform funtion ...');
     value.username = "Salem BEN AMOR";
}

console.log(`student before transform call:${JSON.stringify(student)}`);
transform(student);
console.log(`student after transform call: ${JSON.stringify(student)}`);

/*
  student has changed when affected in the transform function
this behavior is called the parameter is passed by reference
and this is true for object type, arrays.
*/
```

La remontée de variables (hoisting)

Les déclarations de variables (et les déclarations en général) sont traitées avant que n'importe quel autre code soit exécuté. Ainsi, déclarer une variable n'importe où dans le code équivaut à la déclarer au début de son *contexte d'exécution*. Cela signifie qu'une variable peut également apparaître dans le code avant d'avoir été déclarée. Ce comportement est appelé « remontée » (**hoisting** en anglais) car la déclaration de la variable est « remontée » au début de la *fonction* courante ou du contexte global.

```
function faireQuelqueChose() {
  console.log(truc); // undefined
  var truc = 111;
```

```
  console.log(truc); // 111
}

// Correspond en fait à :
function faireQuelqueChose() {
  let truc;
  console.log(truc); // undefined
  truc = 111;
  console.log(truc); // 111
}

// if declared with let then an error raise :
// ReferenceError: Cannot access 'truc' before initialization
// at faireQuelqueChose
```

La « remontée » des déclarations de fonction

Lorsqu'on utilise une déclaration de fonction pour créer une fonction, la définition de la fonction est « remontée ». Il devient donc possible d'utiliser la fonction avant de l'avoir déclarée :

```
remontée(); // affiche "toto" dans la console

function remontée() {
  console.log("toto");
}
```

On notera que les expressions de fonctions ne sont pas remontées :

```
nonRemontée(); // TypeError: nonRemontée is not a function

var nonRemontée = function() {
  console.log("truc");
};
```

Portée d'une fonction

On ne peut pas accéder aux variables définies dans une fonction en dehors de cette fonction : ces variables n'existent que dans la portée de la fonction. En revanche, une fonction peut accéder aux différentes variables et fonctions qui appartiennent à la portée dans laquelle elle est définie. Une fonction définie dans une autre fonction peut également accéder à toutes les variables de la fonction « parente » et à toute autre variable accessible depuis la fonction « parente ».

```
// Les variables suivantes sont globales
 var num1 = 20,
     num2 = 3,
     nom = "Licorne";

 // Cette fonction est définie dans la portée globale
 function multiplier() {
   return num1 * num2;
 }

 multiplier(); // Renvoie 60

 // Un exemple de fonction imbriquée
 function getScore () {
   var num1 = 2,
       num2 = 3;

   function ajoute() {
     return nom + " a marqué " + (num1 + num2);
   }

   return ajoute();
 }

 getScore(); // Renvoie "Licorne a marqué 5"
```

Fonctions imbriquées et fermetures (Closure)

Il est possible d'imbriquer une fonction dans une autre fonction. La portée de la fonction fille (celle qui est imbriquée) n'est pas contenue dans la portée de la fonction parente. En revanche, la fonction fille bénéficie bien des informations de la fonction parente grâce à sa portée. On a ce qu'on appelle une fermeture (closure en anglais). Une fermeture est une expression (généralement une fonction) qui accède à des variables libres ainsi qu'à un environnement qui lie ces variables (ce qui « ferme » l'expression).

Une fonction imbriquée étant une fermeture, cela signifie qu'une fonction imbriquée peut en quelque sorte hériter des arguments et des variables de la fonction parente.

En résumé : La fonction imbriquée ne peut être utilisée qu'à partir des instructions de la fonction parente. La fonction imbriquée forme une fermeture : elle peut utiliser les arguments et les variables de la fonction parente. En revanche, la fonction parente ne peut pas utiliser les arguments et les variables de la fonction fille.

Fermetures (closures)

Les fermetures sont l'une des fonctionnalités les plus intéressantes de JavaScript. Comme on l'a vu précédemment, JavaScript permet d'imbriquer des fonctions et la fonction interne aura accès aux variables et paramètres de la fonction parente. À l'inverse, la fonction parente ne pourra pas accéder aux variables liées à la fonction interne. Cela fournit une certaine sécurité pour les variables de la fonction interne. De plus, si la fonction interne peut exister plus longtemps que la fonction parente, les variables et fonctions de la fonction parente pourront exister au travers de la fonction interne. On crée une fermeture lorsque la fonction interne est disponible en dehors de la fonction parente.

```
var animal = function(nom) {   // La fonction externe utilise un paramètre "nom"
  var getNom = function () {
    return nom;                // La fonction interne accède à la variable "nom" de la fonct
  }
  return getNom;               // Renvoie la fonction interne pour la rendre disponible en d
}

monAnimal = animal("Licorne");

monAnimal();                   // Renvoie "Licorne"
```